RESCUE RATS - SLOVAKIA

RoboCupJunior - Rescue Maze 2026

Movement System

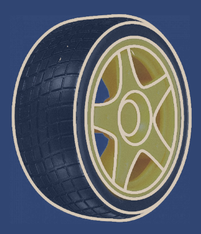
Motors and Motor Bus

JGY-370 Motors: The robot is driven by four 12V JGY-370 DC motors equipped with built-in quadrature encoders for distance counting.

Motor Bus: To handle the high-current demands of the power subsystem without interfering with sensitive logic, the motors are wired into a power bus supplied by a 14.7V LiPo battery. The ESP32 regulates this power via MDD10A Cytron motor drivers, running them at a lower PWM (60-80/255) to prevent overheating.

Wheel Design and Turning

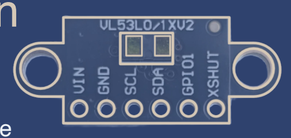
Movement relies on specialized wheels featuring integrated outer rollers. Because these rollers rotate perpendicular to the direction of travel, they reduce the friction and force required to spin in place. Backed by real-time gyroscope tracking, this setup allows the robot to execute 90°and 180° grid turns while maintaining a straight line during forward travel.



Prior Solution: Our initial build utilized standard Arduino Kit Wheels. These suffered from turning resistance, causing wheel slippage, odometry drift, and frequent wall collisions during sharp grid maneuvers.

Obstacle Detection

Wall Detection



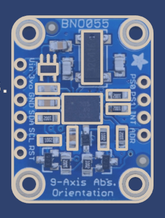
The robot features 8x VL53L1X Time-of-Flight (ToF) laser distance sensors placed one on each corner of the sidewalls. These sensors allow the robot to maintain a safe distance from walls and calculate its alignment. Because all 8 sensors share the same default I2C address, we utilize the hardware XSHUT pins at boot, to dynamically overwrite and reassign unique addresses.

Color of Tiles Detection

A TCS34725 RGB color sensor is mounted to the underside of the front chassis. It constantly scans the floor to identify special tiles (such as black pits, penalty zones, or silver checkpoints), triggering immediate path recalculations.

Detection of Stairs and Ramps

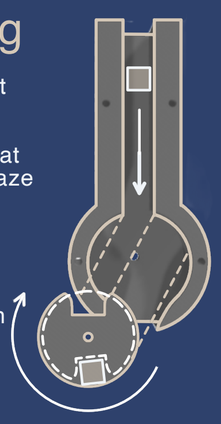
A BNO055 9-axis IMU (combining an accelerometer and gyroscope) tracks the robot's real-time incline. When a ramp or stair configuration is detected, the low-level controller automatically raises the motor PWM output to prevent stalling on the slopes.



Rescue Kits Handling

Dispensing Mechanism: The rescue kit deployment module is entirely modeled in-house and 3D printed. It features a gravity-fed vertical storage magazine that holds the rescue kits securely during maze navigation.

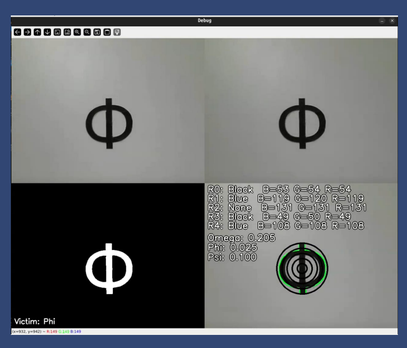
Servo-Driven Actuation: When the mapping logic confirms a valid victim, a 5V servo motor controlled by the ESP32-S3 rotates a notched distribution wheel. This mechanical sweep isolates and drops exactly one rescue kit down the deployment chute onto the tile.

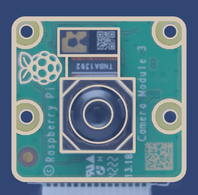


Victim Recognition

Letter Victim Recognition

Using two separates cameras on each side and OpenCV on the Raspberry Pi 5, the vision pipeline adjusts image contrast and saturation before thresholding. It utilizes OpenCV's matchShapes() function to compare runtime contours against pre-made 64x64 .npy reference arrays of Greek letters (Ω- Omega, Φ- Phi, and Ψ- Psi).

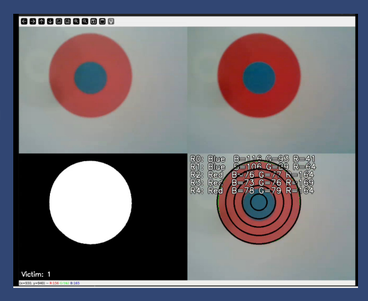


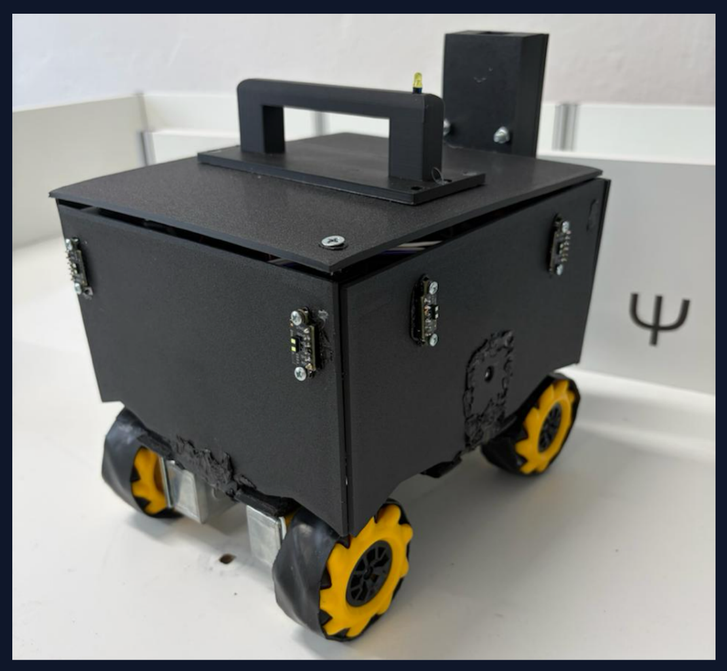


Cognitive Targets Recognition

For cognitive targets, the robot samples the average color values of 5 concentric rings around the largest detected shape contour. Each ring is scored based on its BGR signature. If all 5 rings match the target criteria, their values are compiled into a validated victim score.

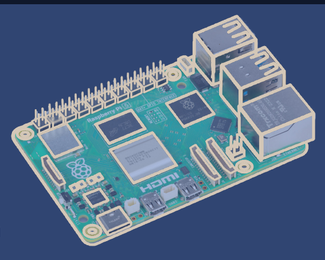
Reliability Filter:
To eliminate false positives from single-frame noise, the camera processes 5 sequential frames per wall. The final victim classification is decided by majority vote across those 5 readings.





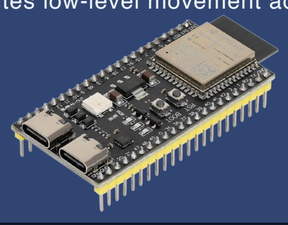
Microcontrollers

Our architecture relies on a clean split between high-level computation and real-time hardware execution.



Raspberry Pi 5: Acts as the main master controller, running the Python software stack responsible for heavy OpenCV image processing, mapping data, and Dijkstra pathfinding decisions.

ESP32 Microcontroller: Acts as the hardware control unit running embedded C++. It directly interfaces with the motor drivers, reads the encoder ticks, manages the I2C sensor bus telemetry, and executes low-level movement adjustments.



Strategy

Our navigation strategy relies on a continuous loop of sensor polling, environmental mapping, and real-time path recalculation. At each tile, the robot averages its sensor pairs to classify tile attributes and record wall structures. It then passes this live map data into a weighted Dijkstra algorithm, which naturally prioritizes exploration by assigning low costs to unvisited tiles while heavily penalizing dangerous zones or blocking black pits entirely.

If the robot experiences a lack of progress or gets trapped, the emergency recovery routine is triggered via a physical switch. The internal coordinates then automatically reset to the nearest safe silver checkpoint tile, allowing the system to seamlessly recalculate a new path and continue the autonomous run without human intervention.

Maze Mapping

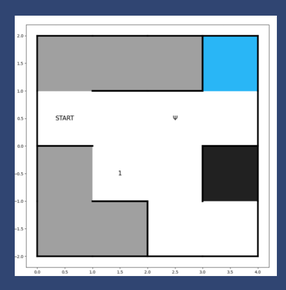
The mapping engine treats the maze as an expanding coordinate dictionary. Each coordinate entry tracks cardinal wall states, tile color, and victim registration to prevent accidental double-drops of rescue kits.

The Weighted Dijkstra Exploration Algorithm

Instead of standard flood-fill, our navigation relies on a weighted version of Dijkstra's Algorithm that dynamically recalculates paths at every exploration cycle iteration based on live map data:

Unvisited Tiles: Cost = 1 (Highly Preferred)
Visited / Silver Tiles: Cost = 2
Incline Tiles (Ramps): Cost = 3
Red Penalty Tiles: Cost = 10
Blue Penalty Tiles: Cost = 50
Black Tiles (Pits): Cost = (Strictly Unpassable)

This cost matrix forces the robot to prioritize efficient, unvisited routes and avoiding high-penalty terrain or dead ends.



visualization of map matrix

```
Output format:
{(x_value, y_value):
    {'Walls':{'direction':True/False
    ..., ..., ...}
    'tile_color':'color',
    'victim':None}}

Output of the top-right tile
{(3,1):
    {'Walls':{'N':True,'S':False,
    'W':True,'E':True},
    'tile_color':'blue',
    'victim':None}}
```

Challenges

I2C Bus Reliability
The initial I2C bus built with Dupont connectors caused connection dropouts between the eight VL53L1X sensors and the IMU.We replaced it with a custom screw-in terminal bus, which improved physical durability and connection stability.

Electromagnetic Interference (EMI)
Motor driver noise often caused sensor communication failures. We mitigated this by wrapping sensitive cables in aluminum shielding and intertwining SDA/SCL lines to create an enclosed field, effectively cancelling out electromagnetic interference.

Crash-Proofing
To prevent data loss during crashes, we utilize Python Context Managers. If the software fails, the system automatically closes the serial port and saves the active map as a .npy file. This ensures the map is preserved and prevents "port busy" errors during rapid restarts.

Meet the Team

Karolína (Mechanical Design):
Managed the mechanical development, including the 3D modeling and printing of the main chassis and the rescue kit deployment mechanism, ensuring all components were refined through multiple design iterations.

Anton (Electronics): Oversaw the assembly and wiring of all electronic systems, such as the distance sensors, color sensor, and IMU, leveraging past experience to ensure reliable connectivity and testing throughout the competition.

Martin (Software):
Developed the entire software stack from scratch, encompassing the maze-solving algorithms, sensor data processing, and victim identification using OpenCV and Python.

Nikita (Documentation):
Managed all project records, including the creation of the competition poster, which proved essential for maintaining clear documentation during building and testing periods.

Our Journey

We started from scratch, building our own practice field and refining our design through many iterations. While we faced challenges, including burned components and failed tests, each mistake taught us something new. This process led us to win the national round, earning our place at the world competition RoboCupJunior 2026 in Incheon, South Korea.

Future Plans

We are focused on rigorous testing, improving software stability and enhancing mechanical durability.
Our goal is to complete a full maze run, identify every victim, and return to the start.

Metrostav Slovakia a.s.

MINISTRY OF EDUCATION, RESEARCH, DEVELOPMENT AND YOUTH OF THE SLOVAK REPUBLIC

Co-funded by the European Union